RAJALAKSHMI
ENGINEERING COLLEGE

An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

BONAFIDE CERTIFICATE

NAME .....Vaishnavi shree. P.....

ACADEMIC YEAR ...2026-27.. SEMESTER .....V.....BRANCH..Comput
an

UNIVERSITY REGISTER No. 2116241401662

Certified that this is the bonafide record of work done by the above student in the

1T23E31-Graphics and multimedia ..... Laboratory during the year 2026 - 2027

14/10/26

Signature of Faculty - in - Charge

Submitted for the Practical Examination held on .....

Internal Examiner

External Examine

Aishwarya P

SUBJECT

DIV. ROLL NO. SCHOOL

TITLE	DATE	TEACHER'S SIGN / REMARKS
Bresenham Line Drawing algorithm	1/8	
Bresenham midpoint circle Drawing algorithm	4/8	
midpoint Ellipse algorithm	9/8	
DDA line Drawing algorithm	2/8	
Bresenham house drawing algorithm	4/8	
Rectangle using Bresenham's algorithm	11/8	
Traffic signal using Bresenham's algorithm	11/8	
Smiling face with eyes nose and mouth using Bresenham's algorithm	25/8	
Olympic logo using Ellipse	25/8	
Butterfly using Ellipse	25/8	
Implementation of scan conversion of polygon and use flood fill algorithm	8/9	
Basic Transformation for 2D shapes for Triangle	22/9	
Basic Transformation for 2D shapes for Rectangle	22/9	
Sutherland Hodgman Polygon clipping algorithm	22/9	
3D shapes Transformation for cube	29/9	
3D Transformations for cone.	29/9	
Rendering 3D objects using polygon and shading Techniques.	10/10	
Multimedia application Integrates Image sound and video in UI	10/10	
Capture video/ audio from Webcam or microphone and Display on Multimedia	10/10.	

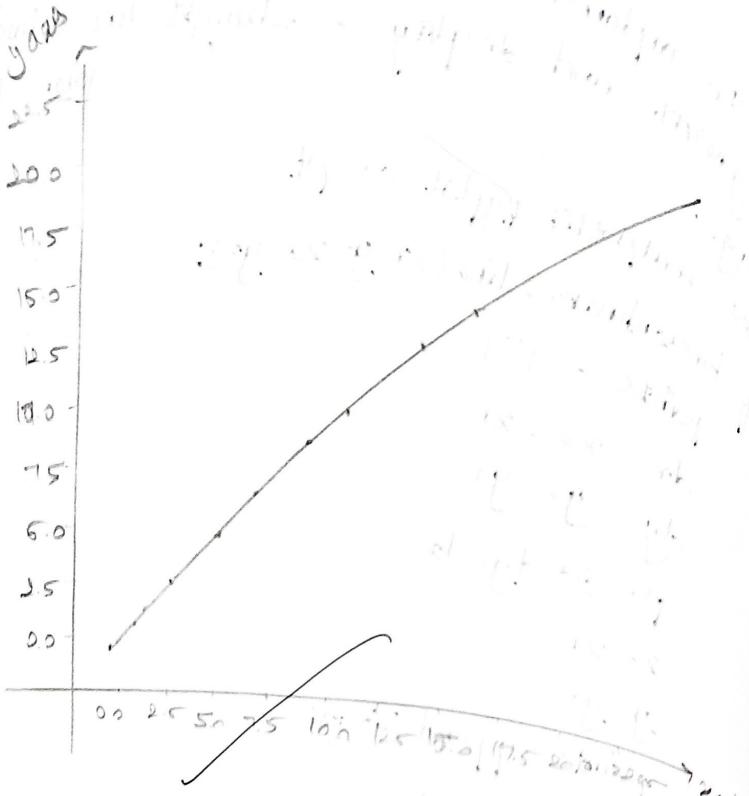
Completed

14/9/26

To implement Bresenham's line drawing algorithm to generate and display a straight line between two given points

Coding:

```
import matplotlib.pyplot as plt
def bresenham_line(x1, y1, x2, y2):
    points = []
    dx = x2 - x1
    dy = y2 - y1
    p = 2 * dy - dx
    x = x1
    y = y1
    points.append((x, y))
    while(x < x2):
        x = x + 1
        if p < 0:
            p = p + 2 * dy
        else:
            y = y + 1
            p = p + 2 * dy - 2 * dx
        points.append((x, y))
    return points
x1 = int(input("Enter x1: "))
y1 = int(input("Enter y1: "))
x2 = int(input("Enter x2: "))
y2 = int(input("Enter y2: "))
```



o/p verified

```
points = bresenham_line (x1,y1,x2,y2)
print ("Points generated :")
for p in points :
    print (p)
x_values = [p[0] for p in points]
y_values = [p[1] for p in points]
plt.plot (x_values, y_values, marker = "o")
plt.title ("Bresenham line drawing algorithm
                                    Circle")
plt.xlabel ("x-axis")
plt.ylabel ("y-axis")
plt.grid()
plt.show()
```

31/7/25

Result.

Thus Bresenham line drawing algorithm was successfully implemented.

Aim: Bresenham's Circle drawing algorithm

To implement midpoint circle algorithm to generate and display circle.

Code:

```
import matplotlib.pyplot as plt
xc = int(input("Enter center x: "))
yc = int(input("Enter center y: "))
r = int(input("Enter radius: "))
x=0
y=r
p=1-r
x_points = []
y_points = []
while x<=y:
    points = [(xc+x, yc+y), (xc-x, yc+y), (xc+x, yc-y),
              (xc-x, yc-y), (xc+y, yc+x), (xc-y, yc+x),
              (xc+y, yc-x), (xc-y, yc-x)]
    for i in points:
        x_points.append(i[0])
        y_points.append(i[1])
    x+=1
    if p<0:
        p=p+2*x+1
```

Exp 6.3
18

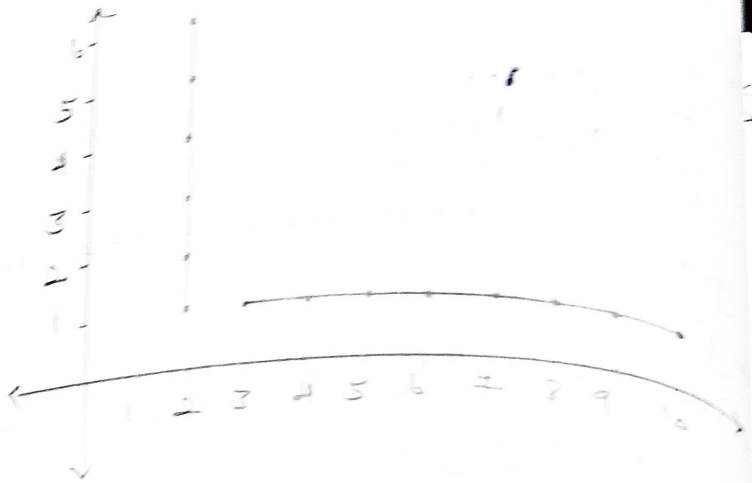
Midpoint Ellipse algorithm

Aim:

To implement midpoint Ellipse algorithm to generate and display ellipse

Code:

```
import matplotlib.pyplot as plt
xc = int(input("Enter center x: "))
yc = int(input("Enter center y: "))
rx = int(input("Enter x radius: "))
ry = int(input("Enter y radius: "))
x = 0
y = ry
rx2 = rx * rx
ry2 = ry * ry
dx = 0
dy = 2 * rx2 * y
p1 = ry2 - rx2 * ry + 0.25 * rx2
x_points = []
y_points = []
while dx < dy:
    points = [(xc+x, yc+y), (xc-x, yc+y), (xc+x, yc-y),
              (xc-x, yc-y)]
    for i in points:
        x_points.append(i[0])
```



```
def dda_line(x0, y0, x1, y1):
    dx = x1 - x0
    dy = y1 - y0
    steps = max(abs(dx), abs(dy))
    if steps == 0:
        return [(x0, y0)]
    x_increment = dx / steps
    y_increment = dy / steps
    x = x0
    y = y0
    points = []
    points.append((round(x), round(y)))
    for _ in range(int(steps)):
        x += x_increment
        y += y_increment
        points.append((round(x), round(y)))
    return points
x_points, y_points = zip(*dda_line(2, 2, 8))
x_points2, y_points2 = zip(*dda_line(1, 1, 5))
```

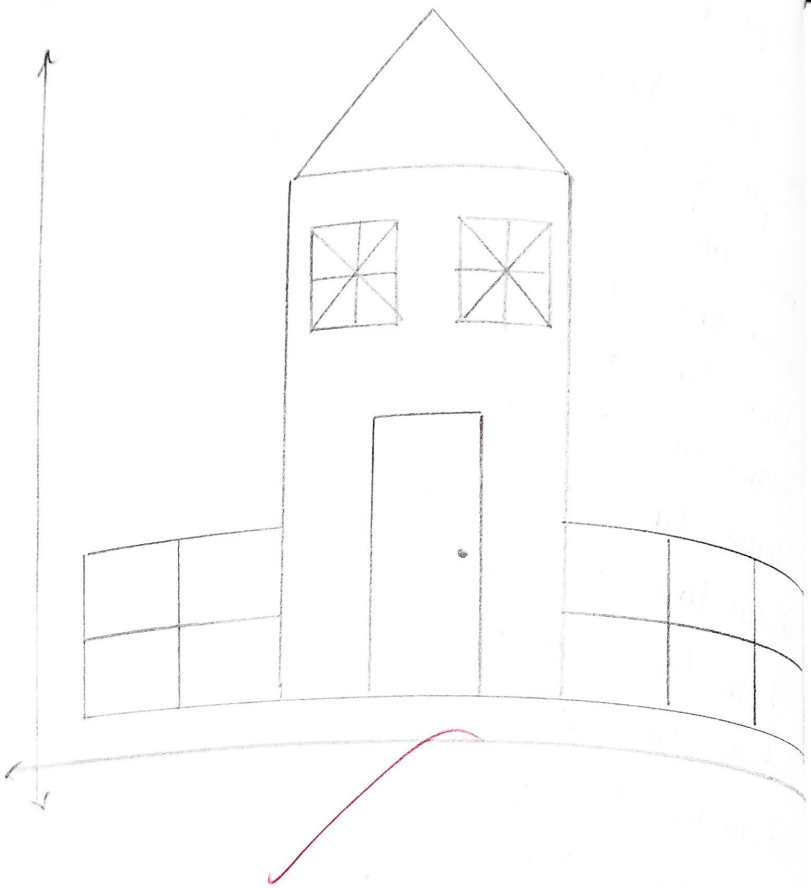
Exp.No:5

Date:

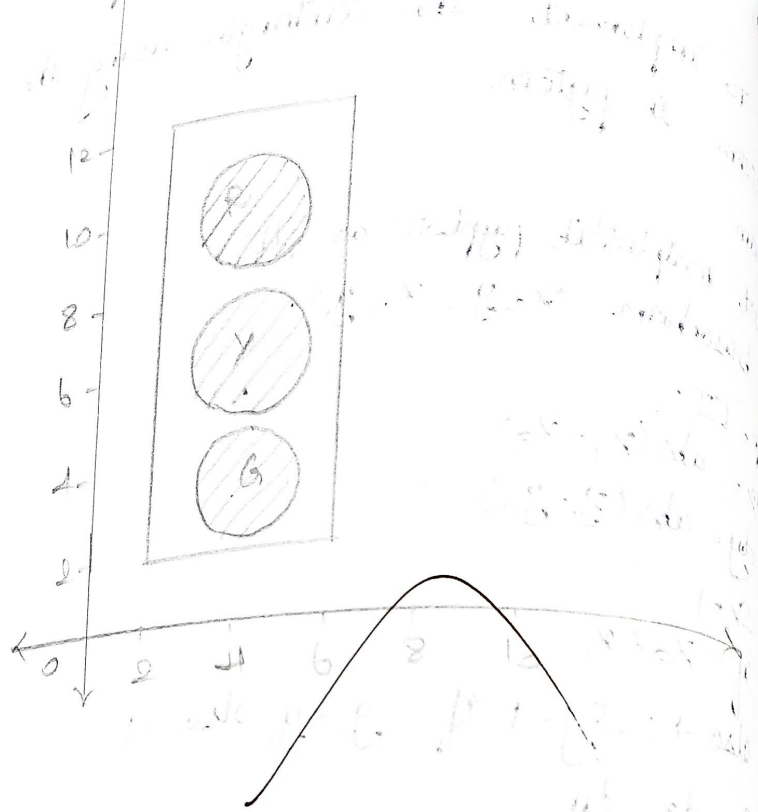
Aim: To implement Bresenham's line drawing algorithm in python using the given coordinates.

Program:

```
import matplotlib.pyplot as plt
def bresenham(x0,y0,x1,y1):
    dx = abs(x1-x0)
    dy = abs(y1-y0)
    sx = 1 if x0<x1 else -1
    sy = 1 if y0<y1 else -1
    err = dx-dy
    points = []
    while True:
        points.append((x0,y0))
        if x0==x1 and y0==y1:
            break
        e2 = 2*err
        if e2>-dy:
            err -= dy
            x0 += sx
        if e2<dx:
            err += dx
```



```
draw_line(3.5, 5.5, 3.5, 4.5)
draw_line(3.5, 4.5, 3.5, 5.5)
draw_line(6.5, 5.5, 7.5, 4.5)
draw_line(6.5, 4.5, 7.5, 5.5)
plt.title("House with windows and fence using
          Bresenham's line algorithm")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.grid(True)   plt.gca().set_aspect('equal')
plt.xticks(range(0,17))
plt.yticks(range(0,12))
plt.xlim(-1,16)
plt.ylim(-1,11)
plt.show()
draw_house()
```



Exp. No: 7 Traffic signal using Bresenham's Algorithm

Date:

Aim:

To implement the program to draw the traffic signal using bresenham's algorithm

Program:

```
import matplotlib.pyplot as plt
from matplotlib.patches import Circle, Rectangle
fig, ax = plt.subplots()
box = Rectangle((3,1), 4, 10, facecolor="black")
ax.add_patch(box)
red = Circle((5,8), 1, facecolor="red", edgecolor="white")
ax.add_patch(red)
yellow = Circle((5,5), 1, facecolor="green", edgecolor="white")
ax.add_patch(green)
ax.set_xlim(0,8)
ax.set_ylim(0,12)
ax.set_aspect("equal")
ax.grid(True)
plt.title("Traffic signal")
plt.show()
```

Result:



Ex No: 7

Date:

Olympic logo using ellipse

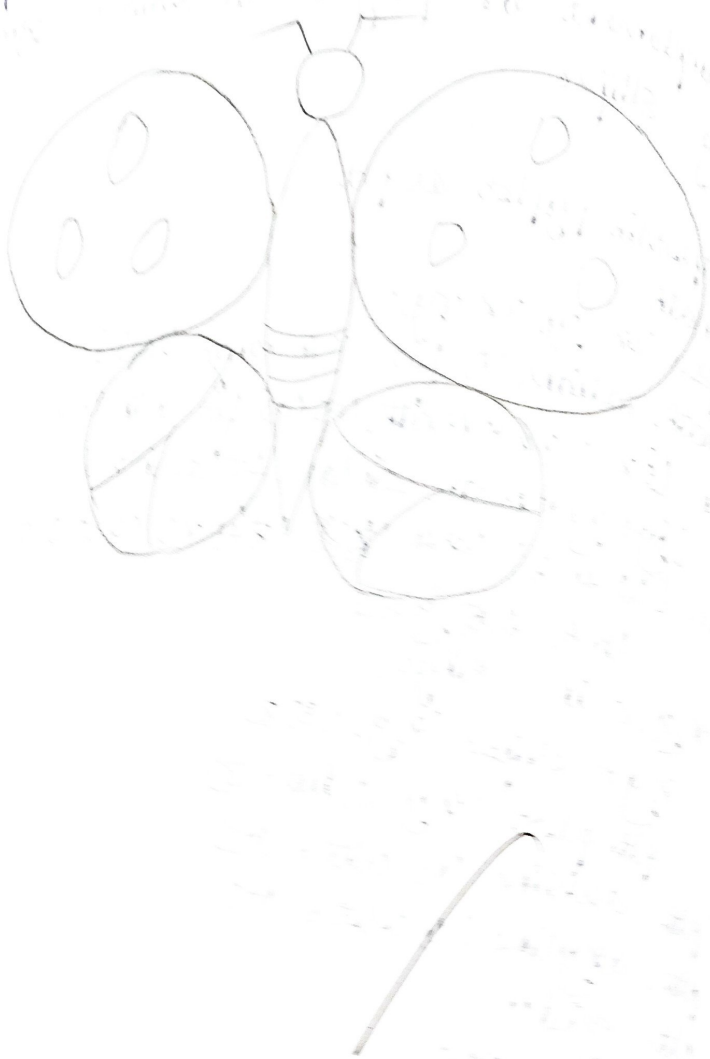
Aim:

To implement the program to draw a olympic logo using Ellipse

Program:

```
import matplotlib.pyplot as plt
import math
def ellipse(x, y, rx, ry):
    t = [math.radians(i) for i in range(360)]
    return [x + rx * math.cos(a) for a in t],
           [y + ry * math.sin(a) for a in t]
rings = [(0, 0, 'b'), (2, 0, 'k'), (4, 0, 'r'), (1, 1, 'y'),
         (3, 1, 'g')]
for x, y, c in rings:
    x, y = ellipse(x, y, 1.5, 0.8)
    plt.plot(x, y, c, lw=3)
plt.axhline(0, color='k')
plt.axvline(0, color='k')
plt.grid()
plt.xlim(0, 6)
plt.ylim(0, 2)
plt.xticks(range(11))
plt.yticks(range(4))
plt.axis('equal')
plt.title("Olympic logo - DDA")
plt.show()
```

output



Ex.No:10
15/2/24

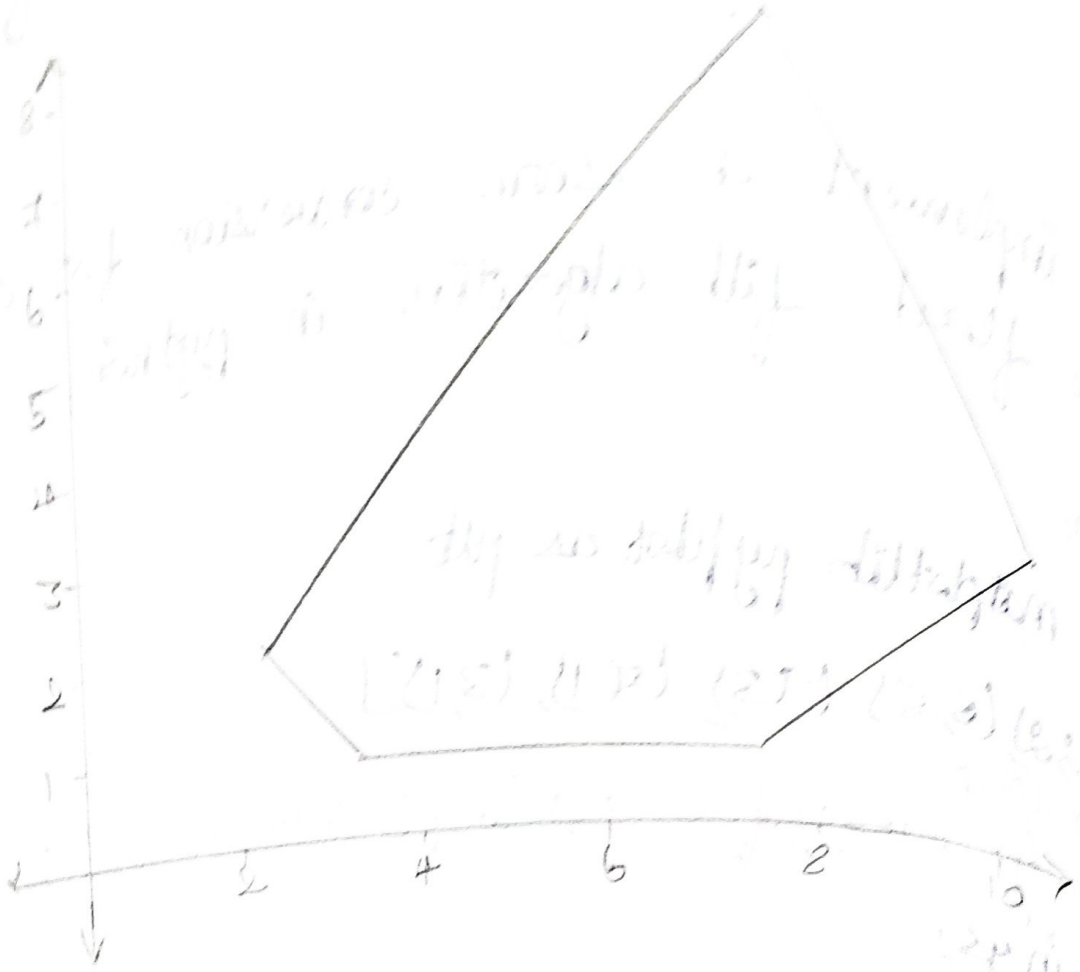
Butterfly using Ellipse

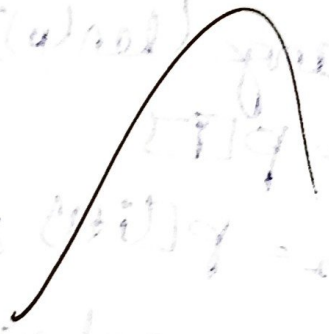
Aim To implement the program to draw a butterfly using Ellipse

Program

```
import matplotlib.pyplot as p
import math
def e(x, y, rx, ry, c='b'):
    for i in range(360):
        t = math.radians(i)
        p.plot(x+rx*math.cos(t), y+ry*math.sin(t), c)
for x in [3.5, 7.5]:
    e(x, 5, 2, 2.5)
    e(x, 2.5, 1.5, 1.5)
    e(5, 4.5, 3, 2.5, 'r')
    e(5, 7.5, 5, 'r')
p.plot([4.8, 4, 3.5], [7.4, 8, 8.3], 'r')
p.plot([5, 2, 6, 6.5], [7.4, 8, 8.5], 'r')
p.axis('equal')
p.grid()
p.show()
```

output





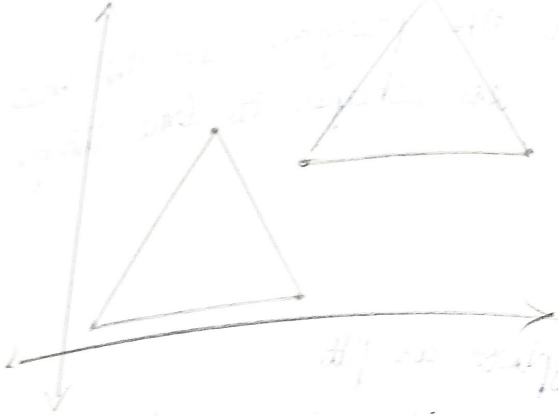
Ex No:12
22/9/25

Basic Transformation for 2D shapes for triangle

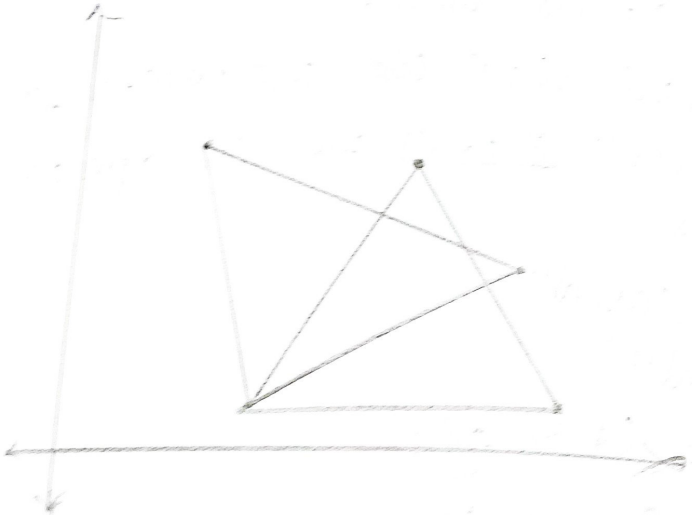
Aim:
To implement the program for the basic transformations for 2D shapes to draw triangle in python

Program:

```
import numpy as np.
import matplotlib.pyplot as plt
shape = np.array([[0,0], [2,0], [1,2], [0,0]])
def plot(ax, s, color, label):
    ax.plot(s[:,0], s[:,1], color=color, marker='o', label)
    ax.fill(s[:,0], s[:,1], color=color, alpha=0.3)
    ax.grid()
    ax.axis('equal')
    ax.legend()
def translate(s, dx, dy):
    return s+[dx, dy]
def rotate(s, angle):
    a=np.radians(angle) R=np.array([[np.cos(a), -np.
                                    [np.sin(a), np.cos(a)]]
    return s@R.T
def scale(s, sx, sy):
```



2D Rotation



2D Scaling



(ii) r = rotate (shape, theta)
fig, ax = plt.subplots()
plot (ax, r, 'blue', 'original') plot (ax, r, 'green', 'rotated')
ax.set_title ("2D Rotation")
plt.show()

(iii) s = scale (shape, 0.5, 0.5)
fig, ax = plt.subplots()
plot (ax, shape, 'blue', 'original') plot (ax, s, 'purple', 'scale')
ax.set_title ("2D Scaling")
plt.show()

Result:

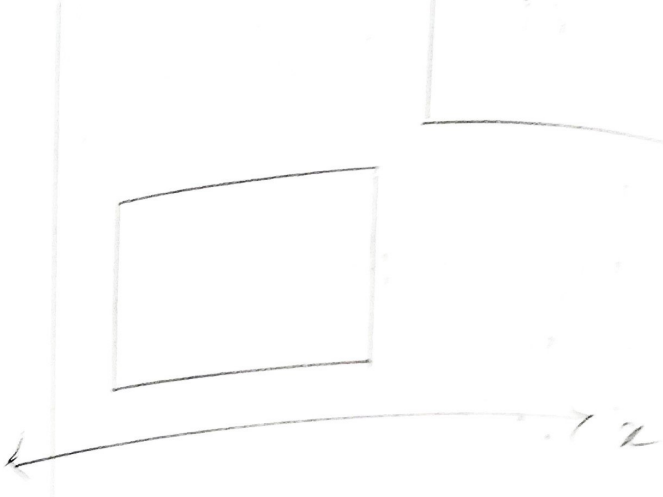
The program to draw the triangle for t
basic transformation is written.

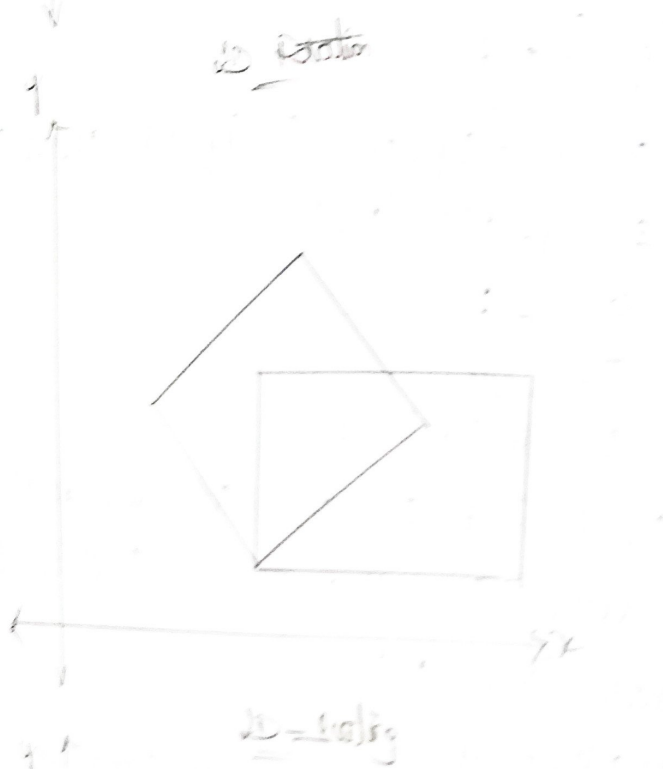
Exp No: 1

Aim: Basic transformation for 2D shapes for rotation.

Aim: To implement the basic transformation shapes in 2D shapes in python program.

```
import numpy as np
import matplotlib.pyplot as plt
def Translate(s, dx, dy):
    return s + [dx, dy]
def rotate(s, angle):
    a = np.radians(angle)
    R = [[np.cos(a), -np.sin(a)], [np.sin(a), np.cos(a)]]
    return s@np.array(R)
def scale(s, sx, sy):
    return s * [sx, sy]
def plot(ax, s, color, label):
    ax.plot(s[:, 0], s[:, 1], color=color, label=label)
    rect = np.array([[
    t = translate(rect
    fig, ax = plt.s
    ax, rect
    title')
```







```
ax.set_title('2D Rotation')
ax.grid()
ax.axis('equal')
ax.legend()
plt.show()
t = np.linspace(0, 2*np.pi, 100)
fig, ax = plt.subplots()
plot(ax, x, y, 'blue', 'original')
ax.set_title('2D Scaling')
ax.grid()
ax.axis('equal')
ax.legend()
plt.show()
```

apsara

Exp No: 14
22/9/21

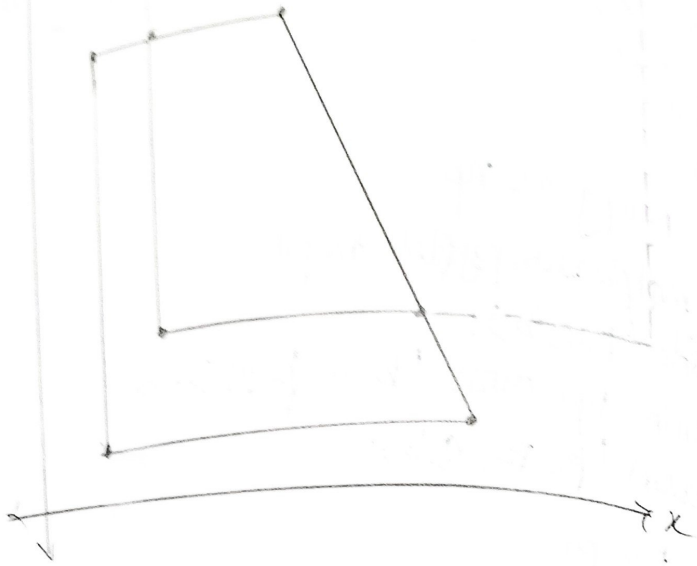
Sutherland - Hodgman polygon clipping algorithm

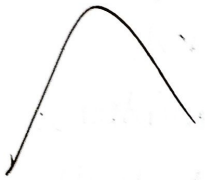
Aim:

To implement the algorithm to clip a subject polygon against a rectangular clipping window using python.

program:

```
import numpy as np
import matplotlib.pyplot as plt
def inside(p,a,b):
    return np.cross(b-a, p-a)>=0
def intersect(p1,p2,a,b):
    d1=p2-p1
    d2=b-a
    c=np.cross(a-p1,d2)/np.cross(d1,d2)
    return p2+c*d1
def clip(subjects,windows):
    output=list(subject)
for i in range(len(window))-1):
    a,b=window[i], window[i+1]
    inp_output=output, []
for j in range(len(inp)):
    p1,p2=inp[j-1], inp[j]
    in1,in2=inside(p1,a,b),inside(p2,a,b)
if in2:
```

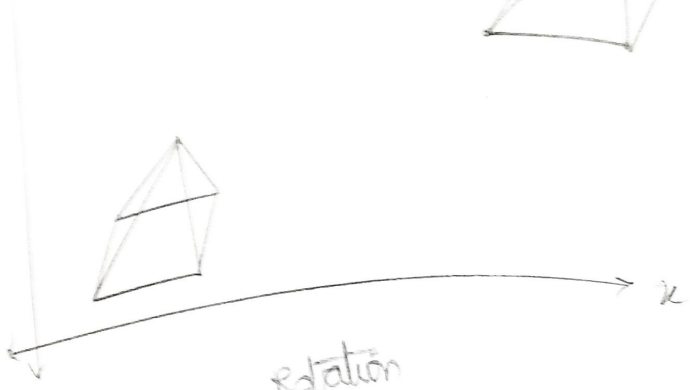




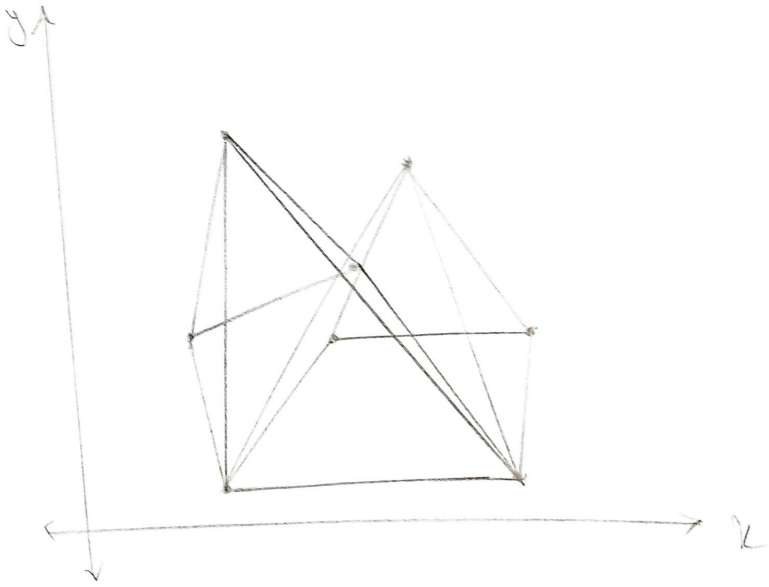
```
subject = np.array([[0.5,0.5],[0.5,0.5],[0.5,0.5],
                    [0.5,0.5]])
window = np.array([[0,0],[0,1],[1,1],[1,0],[0,0]])
result = clip(subject,window)
plt.plot(subject[:,0],subject[:,1],'b-o',label='subject')
if len(result)>0:
    result = np.vstack([result,result[0]])
plt.title('polygon clipping'); plt.xlabel('x'); plt.ylabel('y')
plt.legend()
plt.show()
```

Result:

The program for the sutherland-hodgman polygon clipping algorithm is done.



rotation



Scaling



Date: 26

Aim:

3D Transformations for cube

To implement the program for the 3D shape transformations for cube.

Program

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
V = np.array([[0,0,0],[1,0,0],[1,1,0],[0,1,0],[0.5,0.5,1]])
H = np.c_[V, np.ones(5)]
E = [(0,1),(1,2),(2,3),(3,0),(0,4),(1,4),(2,4),(3,4)]
F = [[0,1,2,3],[0,1,4],[1,2,4],[2,3,4],[3,0,4]]
def plot(T, title, colors):
    W = (T @ H.T).T[:, :3]
    fig = plt.figure(figsize=(8,6))
    ax = fig.add_subplot(111, projection='3d')
for i, c in enumerate(['red','blue']), (W, colors):
    for ij in E:
        ax.add_collection3d(Poly3DCollection([[
            x in f] for f in F], alpha=.2, fac
        ax.set(xlabel='x', ylabel='y', zlabel='z
        plt.show()
        T = np.array([[1,0,0,2],[0,1,0,2],[0,0,
        a = np.pi/4
        T = np.array([[np.cos(a),np.sin(a),0,0
                       [-np.sin(a),np.cos(a),0,0],[0,0,1,0]
```

Rendering 3D objects using Polygon and shading techniques

Aim:

To create and render 3D objects using Polygon modelling and apply flat shading & Basic colour techniques.

Program:

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
import numpy as np
vertices = np.array([[0,0,0],[0,0,1],[0,1,0],[0,1,1],[1,0,0],[1,0,1],
[1,1,0],[1,1,1]])
faces = [[vertices[j] for j in [0,1,2,3]],[vertices[j] for j in
[4,5,6,7]],[vertices[j] for j in [0,1,5,4]]]
colors = ['red','blue','green','yellow','orange','purple']
fig = plt.figure()
ax = fig.add_subplot(111,projection='3d')
cube = Poly3DCollection(faces,facecolors=colors,alpha=0.7)
ax.add_collection3d(cube)
ax.scatter(vertices[:,0],vertices[:,1],vertices[:,2],color='black')
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_xlim(0,1)
ax.set_ylim(0,1)
plt.show()
```

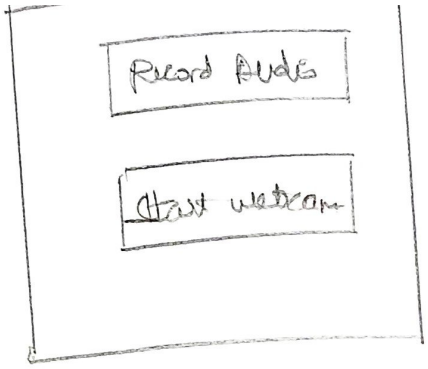
Result:

3D polygonal objects were successfully created and rendered.

```
Multimedia application program
Image, audio and video in it.
aim: To develop a simple multimedia application that
integrates images, video and audio in it.
Program.
import tkinter as tk
from PIL import ImageTk, Image
from playsound import playsound
import cv2
import threading
def image():
    img = image.open("sample_image.jpg").resize((200,200))
    photo = ImageTk.PhotoImage(img)
    panel.config(image=photo)
    panel.image = photo
def audio():
    playsound("sample_audio.mp3")
def video():
    cap = cv2.VideoCapture("sample_video.mp4")
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        cv2.imshow("Video", frame)
        if (cv2.waitKey(25) & 0xFF == ord('q')):
            break
    cap.release()
    cv2.destroyAllWindows()
```

Result:-

A multimedia application was successfully cre
by integrating images sound and video in



10/10/24

Aim:

To develop a program that captures live video and audio from the webcam and microphone.

```
import tkinter as tk
import cv2
import sounddevice as sd
from scipy.io.wavfile import write
import threading
def audio():
    fs = 44100
    print("Recording...")
```

